

Assignment 2

Playfair Cipher

Name: Arnav Singhal

MIS: 612203012

Final Year B. Tech. Comp. Div 1

Batch: B3

1. Introduction

The Playfair cipher is a classical symmetric encryption technique that encrypts pairs of letters (digraphs) rather than single letters. Because it encrypts letter pairs instead of single letters, it resists simple frequency analysis far better than a basic substitution cipher, although it is considered weak by modern standards.

2. Building the Key Square

Both encryption and decryption rely on a 5x5 grid of letters called the key square, built from a chosen keyword:

- Remove duplicate letters from the keyword and write them into the grid, left to right, top to bottom.
- Fill the remaining grid cells with the rest of the alphabet in order, skipping letters already used.
- The letters I and J are treated as the same letter so that the 26-letter alphabet fits into 25 cells.

For example, the keyword "MONARCHY" produces the following key square:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

3. Encryption Process

The plaintext is first split into pairs of letters (digraphs) using these preparation rules:

- If a pair contains two identical letters, an 'X' is inserted between them and the pairing continues.
- If the plaintext has an odd number of letters, an 'X' is appended to the final letter to complete the last pair.

Each plaintext pair is then encrypted by locating both letters in the key square and applying one of three rules:

- Same row: replace each letter with the letter immediately to its right (wrapping to the start of the row if needed).

- Same column: replace each letter with the letter immediately below it (wrapping to the top of the column if needed).
- Different row and column (a rectangle): replace each letter with the letter in its own row but in the other letter's column.

4. Worked Example

Using the key square above (keyword "MONARCHY"), encrypt the plaintext "HELLO WORLD".

Step 1 - split into digraphs: since "LL" repeats, an X is inserted, giving:

HE LX LO WO RL DX

Step 2 - encrypt each pair using the key square (all six pairs happen to form a rectangle):

Pair	Grid positions	Rule applied	Cipher pair
HE	row1,col1 / row2,col0	rectangle rule	CF
LX	row3,col0 / row4,col3	rectangle rule	SU
LO	row3,col0 / row0,col1	rectangle rule	PM
WO	row4,col2 / row0,col1	rectangle rule	VN
RL	row0,col4 / row3,col0	rectangle rule	MT
DX	row1,col4 / row4,col3	rectangle rule	BZ

Step 3 - concatenate the cipher pairs to get the final ciphertext:

Plaintext: HELLO WORLD

Ciphertext: CFSUPMVNMTBZ

5. Decryption Process

Decryption uses the same key square and reverses each rule: same row -> shift left instead of right; same column -> shift up instead of down; rectangle -> the rule is symmetric, so it is applied the same way as encryption. Applying this to "CFSUPMVNMTBZ" with the key square above correctly recovers "HELXLOWORLDX", which reads as "HELLO WORLD" once the padding X letters are removed by the reader.

6. Cryptanalysis Program

The accompanying Python program (playfair_cipher.py) implements both the cipher itself and a simple cryptanalysis (code-breaking) routine that recovers the plaintext from ciphertext alone, without being given the key.

6.1 Encryption / Decryption Functions

The program builds the key square from any keyword and provides encrypt() and decrypt() functions that implement the rules described above, including the digraph preparation step.

6.2 Cryptanalysis Approach: Weak-Key Dictionary Attack

A real attacker rarely has the key, but Playfair keys are, in practice, almost always ordinary English words or names, since that is what people can remember. The program exploits this weakness with a simple, reliable attack:

- A small dictionary of common English words is kept in the program (e.g. MONARCHY, SECURITY, KEYWORD, PASSWORD, and others).
- Every word in the dictionary is tried as the key: the ciphertext is decrypted with each candidate key square.
- Each resulting candidate plaintext is scored using English digraph (letter-pair) frequency statistics - text containing common pairs such as TH, HE, IN, and ER scores higher.
- The candidate with the highest score is returned as the recovered plaintext and keyword.

This is a simplified but genuine cryptanalysis technique: it does not need any prior knowledge of the key and correctly recovers both the keyword and the plaintext whenever the true key appears in the dictionary, which is exactly the scenario demonstrated by the program.

6.3 Code and Output

```
import re
```

```
ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

```
def build_key_square(keyword):
```

```
    keyword = keyword.upper().replace("J", "I")
```

```
    seen = []
```

```
    for ch in keyword:
```

```
        if ch.isalpha() and ch not in seen:
```

```
            seen.append(ch)
```

```
    for ch in ALPHABET:
```

```
        if ch not in seen:
```

```
            seen.append(ch)
```

```
    return seen
```

```
def grid_position(square, letter):
```

```
    idx = square.index(letter)
```

```
    return idx // 5, idx % 5
```

```
def prepare_plaintext(text):
```

```

text = re.sub(r"^[A-Za-z]", "", text).upper().replace("J", "I")
pairs = []
i = 0
while i < len(text):
    a = text[i]
    b = text[i + 1] if i + 1 < len(text) else "X"
    if a == b:
        pairs.append(a + "X")
        i += 1
    else:
        pairs.append(a + b)
        i += 2
if len(pairs[-1]) == 1:
    pairs[-1] += "X"
return pairs

```

```

def encrypt(plaintext, keyword):
    square = build_key_square(keyword)
    pairs = prepare_plaintext(plaintext)
    cipher = ""
    for a, b in pairs:
        ra, ca = grid_position(square, a)
        rb, cb = grid_position(square, b)
        if ra == rb: # same row -> shift right
            cipher += square[ra * 5 + (ca + 1) % 5]
            cipher += square[rb * 5 + (cb + 1) % 5]
        elif ca == cb: # same column -> shift down
            cipher += square[((ra + 1) % 5) * 5 + ca]
            cipher += square[((rb + 1) % 5) * 5 + cb]
        else: # rectangle -> swap columns
            cipher += square[ra * 5 + cb]
            cipher += square[rb * 5 + ca]

```

```
return cipher
```

```
def decrypt(ciphertext, keyword):
```

```
    square = build_key_square(keyword)
```

```
    return decrypt_with_square(ciphertext, square)
```

```
def decrypt_with_square(ciphertext, square):
```

```
    text = re.sub(r"[^A-Za-z]", "", ciphertext).upper()
```

```
    plain = ""
```

```
    for i in range(0, len(text) - 1, 2):
```

```
        a, b = text[i], text[i + 1]
```

```
        ra, ca = grid_position(square, a)
```

```
        rb, cb = grid_position(square, b)
```

```
        if ra == rb:
```

```
            plain += square[ra * 5 + (ca - 1) % 5]
```

```
            plain += square[rb * 5 + (cb - 1) % 5]
```

```
        elif ca == cb:
```

```
            plain += square[((ra - 1) % 5) * 5 + ca]
```

```
            plain += square[((rb - 1) % 5) * 5 + cb]
```

```
        else:
```

```
            plain += square[ra * 5 + cb]
```

```
            plain += square[rb * 5 + ca]
```

```
    return plain
```

```
CANDIDATE_KEYWORDS = [
```

```
    "MONARCHY", "PLAYFAIR", "KEYWORD", "SECURITY", "CRYPTOGRAPHY",
```

```
    "COMPUTER", "NETWORK", "ENCRYPT", "PASSWORD", "GOVERNMENT",
```

```
    "HISTORY", "SCIENCE", "FREEDOM", "JUSTICE", "MESSAGE",
```

```
    "SECRET", "CIPHER", "ALGORITHM", "UNIVERSITY", "LANGUAGE",
```

```
    "MATHEMATICS", "ENGINEERING", "INFORMATION", "COMMUNICATION",
```

```
    "PROTOCOL", "DEFENSE", "STRATEGY", "AUTHORITY", "REPUBLIC",
```

```
"DIAMOND", "ELEPHANT", "MOUNTAIN", "TREASURE", "CASTLE",  
"GARDEN", "LIBRARY", "MIDNIGHT", "SHADOW", "THUNDER",  
]
```

```
COMMON_DIGRAMS = {  
    "TH": 3.56, "HE": 3.07, "IN": 2.43, "ER": 2.05, "AN": 1.99,  
    "RE": 1.85, "ON": 1.76, "AT": 1.49, "EN": 1.45, "ND": 1.35,  
    "TI": 1.34, "ES": 1.34, "OR": 1.28, "TE": 1.20, "OF": 1.17,  
    "ED": 1.17, "IS": 1.13, "IT": 1.12, "AL": 1.09, "AR": 1.07,  
    "ST": 1.05, "TO": 1.04, "NT": 1.04, "NG": 0.95, "SE": 0.93,  
    "HA": 0.93, "AS": 0.87, "OU": 0.87, "IO": 0.83, "LE": 0.83,  
    "VE": 0.83, "CO": 0.79, "ME": 0.79, "DE": 0.76, "HI": 0.76,  
    "RI": 0.73, "RO": 0.73, "IC": 0.70, "NE": 0.69, "EA": 0.69,  
    "RA": 0.69, "CE": 0.65, "LI": 0.62, "CH": 0.60, "LL": 0.58,  
    "BE": 0.58, "MA": 0.56, "SI": 0.55, "OM": 0.55, "UR": 0.54,  
    "CA": 0.52, "EL": 0.51, "TA": 0.50, "LA": 0.49, "NS": 0.49,  
    "DI": 0.48, "FO": 0.48, "HO": 0.48, "PE": 0.47, "EC": 0.46,  
    "PR": 0.45, "NO": 0.45, "CT": 0.43, "US": 0.43, "AC": 0.42,  
    "OT": 0.42, "IL": 0.42, "TR": 0.42, "LY": 0.41, "NC": 0.40,  
    "WA": 0.40, "EM": 0.39, "WI": 0.39, "WH": 0.39, "PA": 0.37,  
    "HT": 0.37, "EE": 0.37, "GE": 0.37, "SO": 0.37, "OW": 0.36,  
    "EV": 0.35, "UN": 0.35, "UT": 0.35, "SA": 0.34, "AD": 0.34,  
    "AM": 0.34, "GA": 0.33, "TS": 0.33, "EI": 0.32, "NI": 0.32,  
    "MO": 0.32, "ID": 0.31, "GH": 0.31, "AB": 0.31, "NA": 0.30,  
    "SU": 0.30, "PI": 0.29, "OP": 0.29, "SH": 0.29, "KE": 0.29,  
    "OL": 0.29, "IF": 0.28, "IE": 0.28, "AY": 0.28, "WO": 0.28,  
}
```

```
def score_text(text):  
    score = 0.0  
    for i in range(len(text) - 1):  
        pair = text[i:i + 2]
```

```

        score += COMMON_DIGRAMS.get(pair, 0.01)
    return score

def dictionary_attack(ciphertext, wordlist=CANDIDATE_KEYWORDS):
    results = []
    for word in wordlist:
        candidate_plain = decrypt(ciphertext, word)
        candidate_score = score_text(candidate_plain)
        results.append((word, candidate_plain, candidate_score))

    results.sort(key=lambda r: r[2], reverse=True)
    return results

if __name__ == "__main__":
    keyword = "MONARCHY"
    plaintext = ("MEET ME AT THE HIDDEN GARDEN BEHIND THE OLD LIBRARY "
                "AFTER MIDNIGHT AND BRING THE SEALED ENVELOPE WITH YOU")

    print("PLAYFAIR CIPHER - ENCRYPTION")
    print("Keyword :", keyword)
    print("Plaintext :", plaintext)

    ciphertext = encrypt(plaintext, keyword)
    print("Ciphertext:", ciphertext)

    decrypted = decrypt(ciphertext, keyword)
    print("Decrypted (with correct key):", decrypted)

    print()
    print("CRYPTANALYSIS")
    print("Attacking ciphertext:", ciphertext)

```

```

ranked = dictionary_attack(ciphertext)

print()
print("Top 3 guesses (keyword : fitness score : decrypted text):")
for word, text, sc in ranked[:3]:
    print(f" {word:12s} {sc:7.2f} {text}")

best_word, best_text, best_score = ranked[0]
print()
print("Recovered keyword :", best_word)
print("Recovered plaintext:", best_text)

```

Output:

```

arnav@arnav-HP-Laptop-15s-fq5xxx:~/Desktop/COEP/CNS/Assignment2$ python3 playfair_cipher.py
PLAYFAIR CIPHER - ENCRYPTION
Keyword   : MONARCHY
Plaintext : MEET ME AT THE HIDDEN GARDEN BEHIND THE OLD LIBRARY AFTER MIDNIGHT AND BRING THE SEALED ENVELOPE WITH YOU
Ciphertext: CLKLCLRSPDFCKBCKYQRMCKAYFCGAKZCFMPCTSIMRND0ILKMOKBAGFYSTRRYDAGAKQCFLIMSKCGMUFPMFLFXGPDHNVZ
Decrypted (with correct key): MEETMEATTHEHIDDENGARDENBEHINDTHEOLDLIBRARYAFTERMIDNIGHTANDBRINGTHESEALEDENVELOPEWITHYOUX

CRYPTANALYSIS
Attacking ciphertext: CLKLCLRSPDFCKBCKYQRMCKAYFCGAKZCFMPCTSIMRND0ILKMOKBAGFYSTRRYDAGAKQCFLIMSKCGMUFPMFLFXGPDHNVZ

Top 3 guesses (keyword : fitness score : decrypted text):
MONARCHY      68.77  MEETMEATTHEHIDDENGARDENBEHINDTHEOLDLIBRARYAFTERMIDNIGHTANDBRINGTHESEALEDENVELOPEWITHYOUX
PROTOCOL      29.14  PEFEPEPTNTLKPHEZEXSPNZEDWKPAREUPKFVTOIDNPSATHEFQPHERAIVNTTWBLARHUPKDFUQEZFNMKVFLWHTLGQZY
PASSWORD      23.86  RQFTRQBPARIRHEEIQIDLEIWIWIRDVETRIAEQWHLDMBWKFTAHEVDIUPBCUAVDVITRIQFNAIEDGLRALFRVHARBHUY

Recovered keyword : MONARCHY
Recovered plaintext: MEETMEATTHEHIDDENGARDENBEHINDTHEOLDLIBRARYAFTERMIDNIGHTANDBRINGTHESEALEDENVELOPEWITHYOUX
arnav@arnav-HP-Laptop-15s-fq5xxx:~/Desktop/COEP/CNS/Assignment2$ █

```

6.3 Demonstration Result

When run, the program encrypts a sample message with the keyword "MONARCHY", then attacks only the resulting ciphertext (without using the keyword). The dictionary attack correctly identifies "MONARCHY" as the key, its digraph score is clearly the highest among all candidates, and the recovered plaintext exactly matches the original message. This confirms both the correctness of the encryption/decryption logic and the effectiveness of the cryptanalysis approach against weak, dictionary-based keys.

7. Conclusion

The Playfair cipher improves on simple substitution ciphers by encrypting letter pairs, which flattens single-letter frequency patterns. However, its security still depends entirely on the secrecy and unpredictability of the key. As demonstrated by the accompanying program, if a human-memorable word is used as the key, the cipher can be broken quickly using nothing more than a small dictionary and basic English letter-pair statistics.